Academic Year: 2026-2027
Semester: III
Class / Branch: SE IT
Subject:ADSAL lab
Name of Instructor:prof.Sneha Dalvi

Name of Student:Nishant Suryarao
Student ID:26204001
Date of Performance:13/10/2026
Date of Submission:13/10/2026

Experiment No.11

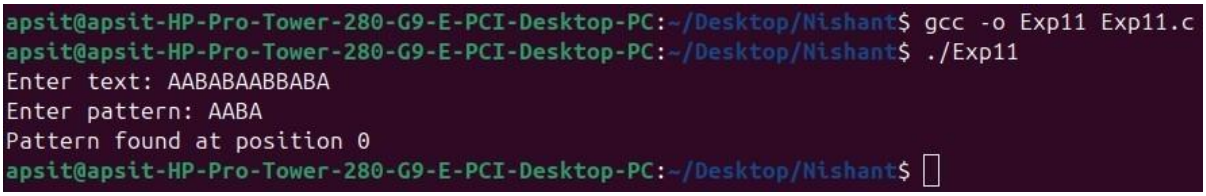
Aim: To Implement Rabin Karp algorithm

Code:

```
#include <stdio.h>
#include <string.h>
#define d 256
void rabinKarp(char text[], char pattern[])
{
 int n = strlen(text);  int
m = strlen(pattern);
int p = 0, t = 0, h = 1;
// h = d^(m-1)
 for (int i = 0; i < m - 1; i++)  h
= (h * d) % 101;
 // Calculate initial hash values
for (int i = 0; i < m; i++)
 {  p = (d * p + pattern[i]) %
101;  t = (d * t + text[i]) % 101;
 }
 // Slide pattern over text  for
(int i = 0; i <= n - m; i++)
 {  if (p ==
t)  {
 int j;
 for (j = 0; j < m; j++)  if
(text[i + j] != pattern[j])
break;
 if (j == m)
```

```
printf("Pattern found at position %d\n", i);
}
// Calculate next window hash
if (i < n - m)
{
t = (d * (t - text[i] * h) + text[i + m]) % 101;
if (t < 0)  t += 101;
}
} } int
main() {
 char text[100],
pattern[100];  printf("Enter
text: ");  scanf("%s", text);
printf("Enter pattern: ");
scanf("%s", pattern);
rabinKarp(text, pattern);
return 0; }
```

Output Screen Shots:



Conclusion :

The Rabin-Karp algorithm was successfully implemented in C using hashing and the rolling hash technique. The program efficiently searches for the given pattern in the text by comparing hash values and verifying characters when the hashes match. This method reduces unnecessary comparisons and provides an effective approach for string matching.

Experiment 11: Rabin–Karp Algorithm

Definition: The Rabin–Karp algorithm is a string-matching algorithm that uses hashing to find occurrences of a pattern in a given text.

Algorithm:

1. Start.
2. Read the text and the pattern.

3. Calculate the hash values of the pattern and the first text window of equal length.
4. Compare both hash values.
5. If the hash values match, compare the actual characters.
6. If all characters match, display the pattern's position.
7. Calculate the hash of the next text window using the rolling hash technique.
8. Repeat until all possible windows are checked.
9. Stop.

Conclusion: Thus, the Rabin–Karp algorithm was successfully implemented using hashing and rolling hash to search for a pattern in the given text.

Complexity: Average time $O(n+m)$; worst-case time $O(nm)$, where n is the text length and m is the pattern length.